

Clase Dentadura

Implementar en Java una clase que permitirá gestionar la situación de cada pieza dental de un paciente adulto. Un diente puede tener los siguientes estados: Bien, extraído, empastado, prótesis.

Una dentadura está compuesta de 32 piezas (16 arriba y 16 abajo), que se identifican por un número concreto (mirar la imagen).

Métodos de la clase Dentadura (al menos):



- Dentadura(): constructor vacío que crea la dentadura estando todos los dientes con estado 'Bien', usando para ello una tabla o matriz para almacenar los dientes.

- Dentadura(int[] cambiar, String estado): todos los dientes están en buen estado salvo los incluidos en el vector argumento, que se ponen como se indique en el segundo argumento

- getter para un diente

- setter para un diente y un estado

- sobrecargar el setter anterior (mismo nombre) para varios dientes y un estado (como el segundo constructor)

- Implementar el método equals que compara dos dentaduras

- void mostrar()

Muestra por pantalla el estado de la dentadura, sustituyendo cada pieza por la inicial de su estado, salvo el caso de estar 'Extraído' que se sustituye por un espacio en

blanco. Por ejemplo:

```
1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
8 7 6 5 4 3 2 1 1 2 3 4 5 6 7 8
| | | | | | | | | | | | | | |
B B B   P P B B B B B B B B   B
B B B B B B B B   B B B B B E
| | | | | | | | | | | | | | |
4 4 4 4 4 4 4 4 3 3 3 3 3 3 3 3
8 7 6 5 4 3 2 1 1 2 3 4 5 6 7 8
```

Métodos de la clase Diente (al menos):

- Diente(int id, String estado): constructor

- getters

- setters

Main:

Crear una dentadura usando el segundo constructor, modificar algunos dientes y mostrar por pantalla el estado de todos los dientes (usando el método mostrar()). Obtener también un diente cualquiera y sacar por pantalla su información usando los getters.

Nota.- como siempre, tened cuidado con la reutilización de código, legibilidad, comentarios...

Para facilitaros el trabajo, os doy dos métodos que os ayudarán a ubicar un diente según su id:

```
private int maxilar(int id)
//devuelve el maxilar (0 -> superior 1-> inferior) para un determinado id
{
    if(id<30)
        return 0;
    else
        return 1;
}

private int posicion(int id)
//devuelve la ubicación (del 0 al 15) para un determinado id
{
    if(maxilar(id) == 0) //estamos en el superior
    {
        if(id < 19) //estamos a la derecha [18...11]
        {
            return 7 - (id-11);
        }
        else //estamos a la izquierda [21...28]
        {
            return 8 + (id-21);
        }
    }
    else //estamos en el inferior
    {
        if(id > 40) //estamos a la derecha [48...41]
        {
            return (48-id);
        }
        else //estamos a la izquierda [31...38]
        {
            return 8 + (id-31);
        }
    }
}
```